JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번 202212205 이름: **이상현** 제출일: 26/10/28

서비스 이름 **게시판**

기술 조합과 버전 React + Vite / Node.js + Express / SQLite

GitHub 저장소: https://github.com/lsanghyl/JWT-Assignment

소스 코드 기준: main 브랜치 (4427582fc6a130260b812750725586819e19df94

)

프론트엔드는 React, 백엔드는 Node.js Express 를 선택하였다. React 는 사용자 로그인 상태와 게시글 데이터를 상태로 관리하고, 사용자 동작에 따라 화면을 갱신하기에 적합하다고 판단하였다. Express 는 HTTP 요청을 처리하는 구조가 비교적 간단하고, JWT 인증 및 작성자 권한 검사를 API 와 연결하여 구현하기에 용이하였다.

제공된 시작 코드에는 게시판 CRUD 기능과 기본 화면이 구현되어 있었으며, 이를 바탕으로 JWT 발급과 검증, 로그인 상태 관리, 인증 오류 처리, 작성자 권한 검사를 추가하였다. 특히 사용자 인증과 게시글 작성자 권한을 구분하여 서버에서 검증하도록 구현하였다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 게시글 목록 및 상세 조회	완료	backend-express/src, frontend-react/src/main.jsx	
로그인 및 JWT 발급	완료	backend-express/src/auth.js, frontend-react/src/api.js	
JWT 인증 및 내 정보 조회	완료	backend-express/src/auth.js	
본인 글 작성·수정·삭제	완료	backend-express/src/auth.js, frontend-react/src/main.jsx	
토큰 만료·로그아웃·오류 표시	완료	frontend-react/src/api.js, main.jsx	

기능	완료 여부	관련 파일	비고 또는 제한
SQLite 데이터 유지	완료	backend-express/data/board.sqlite	

구조와 변경 내역

본 프로젝트는 React 프론트엔드, Express 백엔드, SQLite 데이터베이스로 구성하였다. React 는 사용자에게 게시판 화면을 제공하고, Express 는 API 요청 처리와 JWT 인증 및 작성자 권한 검사를 담당한다. 게시글 데이터는 SQLite 파일 데이터베이스에 저장된다.

전체 시스템 구조

[사용자 / 웹 브라우저]

|

v

[React 프론트엔드]

- 로그인 및 로그아웃
- 게시글 목록과 상세 화면
- 게시글 작성·수정·삭제
- JWT 메모리 보관

|

| HTTP 요청

| Authorization: Bearer JWT

v

[Express 백엔드]

- 로그인 정보 검증
- JWT 발급 및 검증
- 사용자 ID 확인
- 게시글 작성자 권한 검사

|

v

[SQLite 데이터베이스]

- 게시글 데이터 저장 및 조회

로그인 요청 흐름

사용자가 React 로그인 화면에서 이메일과 비밀번호를 입력하면 프론트엔드는 POST /api/auth/login 요청을 전송한다.

Express 서버는 입력받은 계정 정보를 확인하고, 인증에 성공하면 사용자 ID 를 포함한 JWT 를 발급한다. 프론트엔드는 반환된 JWT 와 사용자 정보를 메모리의 session 변수에 저장한다.

이후 인증이 필요한 API 를 호출할 때 JWT 를 Authorization: Bearer <token> 헤더에 포함하여 전송한다. 토큰은 localStorage 나 sessionStorage 에 저장하지 않으며, 페이지를 새로고침하거나 로그아웃하면 메모리에 저장된 로그인 상태가 해제된다.

게시글 수정 요청 흐름

사용자가 게시글 수정을 요청하면 React 는 저장된 JWT 를 포함하여 PUT /api/posts/:id 요청을 전송한다.

Express 는 JWT 의 서명과 만료 여부, 사용자 정보를 검증한 후 요청한 사용자의 ID 를 확인한다. 이 과정은 검증된 사용자의 신원을 이후 게시글 처리 과정에 전달하는 요청 주체 연결점에 해당한다.

이후 게시글에 저장된 작성자 ID 와 인증된 사용자 ID 를 비교한다. 이 비교는 권한 경계 대조에 해당하며, 두 ID 가 일치하면 수정 요청을 허용하고 일치하지 않으면 403 FORBIDDEN 을 반환한다.

이처럼 프론트엔드에서 수정·삭제 버튼을 표시하는 것과 별개로 서버에서 권한을 다시 검사하도록 구성하였다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend-express/src/auth.js / issueToken()	JWT 발급 관련 시작 코드	HS256 JWT 발급, 사용자 ID 와 만료 시간 등 클레임 설정	backend-express/src/auth.js / issueToken()
backend-express/src/auth.js / currentUserId()	인증 검증 TODO	Bearer 토큰 추출, JWT 검증, 사용자 ID 확인, 인증 오류 처리	backend-express/src/auth.js / currentUserId()
backend-expres/src/auth.js / assertOwner()	권한 검사 TODO	로그인 사용자와 게시글 작성자 비교, 타인 글 변경 시 403 반환	backend-express/src/auth.js / assertOwner()

기존에 제공된 게시글 CRUD 기능과 React 화면 구조를 최대한 유지하면서 인증과 권한 관련 기능을 추가하였다. 이를 통해 기존 게시판 동작을 유지하고 JWT 인증을 연결하는 데 집중하였다.

코드 작성 및 수정 출처

제공된 React 와 Express 시작 코드를 기반으로 구현하였으며, JWT 인증과 권한 검사, 프론트엔드 연동 과정에서 ChatGPT 를 활용하였다.

ChatGPT 가 제안한 코드와 설명을 참고하여 TODO 함수를 구현하였고, PowerShell 을 이용한 API 요청 및 브라우저 테스트로 동작을 확인하였다. 기존 제공 코드와 AI 의 도움을 받아 작성한 부분을 구분하였으며, 구체적인 활용 내용과 검증 과정은 05 장에서 설명한다.

로컬 실행 기록

개발 환경

- **운영체제**: Windows 11

- **백엔드**: Node.js v24.21.0, Express
- **프론트엔드**: React, Vite
- **데이터베이스**: SQLite
- **테스트 도구**: PowerShell, Git Bash, **웹 브라우저**

백엔드 실행

백엔드 폴더에서 필요한 패키지를 설치한 후 JWT **비밀키를 환경변수로 설정하였다.**

```
cd backend-express
npm install
$env:JWT_SECRET = node -e "process.stdout.write(require('crypto').randomBytes(32).toString('hex'))"
npm run dev
```

JWT **비밀키는** 32 **바이트의 무작위 값을** 64 **자리** 16 **진수 문자열로 생성하였다. 환경변수가 설정되지 않은 상태에서는 서버가 실행되지 않았으며, 비밀키를 올바르게 설정한 뒤 서버를 실행할 수 있었다.**

백엔드는 http://localhost:8000 **에서 실행하였으며,** /api/health **와 인증** API **를 이용하여 정상 동작을 확인하였다.**

프론트엔드 실행

별도의 터미널에서 다음 명령어로 React **개발 서버를 실행하였다.**

```
cd frontend-react
npm install
npm run dev
```

브라우저에서 http://localhost:5173 **에 접속하여** Alice **와** Bob **계정으로 로그인하고, 게시글 조회 및 작성자별 수정·삭제 버튼 표시 기능을 확인하였다.**

SQLite **데이터 저장 및 로컬 실행 검증**

게시글 데이터는 SQLite **파일 데이터베이스를 이용하여 저장하도록 구성하였다. 데이터베이스 파일 경로는** backend-express/data/board.sqlite **이다.**

React **프론트엔드에서** npm run build **로 빌드한 후** npm run preview **를 실행하여** 4173 **번 포트에서 접속하였다.** Express **백엔드는** 8000 **번 포트에서 실행하였으며, 게시글 목록 조회,** Alice **계정 로그인, 게시글 작성 및 조회가 정상적으로 동작하는 것을 확인하였다.**

또한 서버를 재실행한 이후에도 이전에 작성한 게시글이 그대로 남아 있는 것을 확인하였다. 이를 통해 SQLite **파일 데이터베이스를 통한 게시글 데이터 유지 기능이 정상적으로 동작함을 검증하였다.**

실행 증거: 보고서 04 **장의 테스트 결과 및 캡처 자료에 첨부 예정.**

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: backend-express/src/auth.js / issueToken()

코드 발췌

```
export function issueToken(user) {
  return jwt.sign({}, jwtConfig.key, {
    algorithm: "HS256",
    subject: String(user.id),
    expiresIn: jwtConfig.tokenTtl,
    issuer: jwtConfig.issuer,
    audience: jwtConfig.audience,
  });
}
```

동작과 응답 코드 설명

사용자가 로그인하면 서버에서 계정 정보를 확인한 후, 인증된 사용자 정보를 issueToken() 함수에 전달하여 JWT 를 발급한다.

JWT 는 jsonwebtoken 라이브러리의 jwt.sign() 함수를 이용하여 생성하며, 서명 알고리즘은 HS256 을 사용한다. 토큰에는 사용자 ID 를 나타내는 sub, 만료 시간을 나타내는 exp, 발급자를 나타내는 iss, 대상 서비스를 나타내는 aud 클레임이 포함된다.

토큰 유효 시간과 서명에 사용되는 비밀키는 jwtConfig 에서 관리하도록 구성하였다. 이를 통해 비밀키를 소스 코드에 직접 작성하지 않고 환경변수로 관리할 수 있도록 하였다.

로그인 성공 시 발급된 JWT 를 프론트엔드에 전달하며, 프론트엔드는 이후 인증이 필요한 요청에 해당 토큰을 사용한다.

로그인 API 의 HTTP 응답 처리는 이 함수를 호출하는 라우터에서 담당하므로, 로그인 성공과 계정 정보 오류에 대한 응답 코드는 해당 라우터를 기준으로 확인한다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: frontend-react/src/api.js / login(), authorizationHeaders(), request(), clearSession()

코드 발췌

```
let session = { accessToken: "", user: null };

export function clearSession() {
```

```
  session = { accessToken: "", user: null };
}

function authorizationHeaders() {
  if (!session.accessToken) {
    return {};
  }
  return {
    Authorization: `Bearer ${session.accessToken}`,
  };
}

export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });

  session = {
    accessToken: data.accessToken,
    user: data.user,
  };

  return data.user;
}
```

동작과 응답 코드 설명

프론트엔드에서는 로그인 상태를 관리하기 위해 session 변수를 사용하였다. 해당 변수에는 서버에서 발급받은 JWT 인 accessToken 과 로그인한 사용자의 정보인 user 를 저장한다.

사용자가 이메일과 비밀번호를 입력하면 login() 함수에서 POST /api/auth/login 요청을 전송한다. 서버에서 로그인에 성공하면 반환된 JWT 와 사용자 정보를 session 에 저장하고, 사용자 정보를 반환하여 화면에 로그인 상태를 표시하도록 하였다.

JWT 는 브라우저의 localStorage 나 sessionStorage 가 아닌 JavaScript 메모리에만 저장한다. 따라서 페이지를 새로고침하면 로그인 상태가 초기화되며, clearSession() 함수를 호출하여 로그아웃할 때도 저장된 토큰과 사용자 정보가 제거된다.

authorizationHeaders() 함수는 저장된 JWT 가 있는지 확인하고, 토큰이 존재하면 Authorization: Bearer <token> 형식의 헤더를 생성한다. 생성된 헤더는 공통 request() 함수에서 API 요청에 추가된다.

또한 request() 함수에서는 서버 응답이 401 인 경우 clearSession()을 호출하여 로그인 상태를 초기화한다. 이를 통해 토큰이 만료되거나 유효하지 않은 경우 사용자가 이전 로그인 상태를 계속 유지하지 않도록 하였다.

주요 HTTP 응답 및 처리

- 로그인 성공: HTTP 200, JWT 및 사용자 정보 저장
- 로그인 정보 오류: HTTP 401, 서버 오류 메시지 표시
- 토큰 누락 또는 만료: HTTP 401, 로그인 상태 초기화
- 정상적인 인증 요청: 저장된 JWT 를 Bearer 헤더에 포함하여 전송

실제 테스트에서 Alice 와 Bob 계정으로 로그인할 수 있었으며, JWT 만료 후 보호된 API 를 호출했을 때 로그인 상태가 해제되는 것을 확인하였다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: backend-express/src/auth.js / currentUserId()

코드 발췌

```
const authorization = req.headers.authorization;

if (
  typeof authorization !== "string" ||
  !/^Bearer [^₩s]+$/.test(authorization)
) {
  throw new ApiError(401, "TOKEN_REQUIRED",
    "Bearer 토큰이 필요합니다.");
}

const token = authorization.slice(7);

const payload = jwt.verify(token, jwtConfig.key, {
  algorithms: ["HS256"],
  issuer: jwtConfig.issuer,
  audience: jwtConfig.audience,
});
```

동작과 응답 코드 설명

currentUserId() 함수는 클라이언트가 전송한 JWT 를 검증하고, 인증된 사용자의 ID 를 반환하는 역할을 한다.

먼저 HTTP 요청의 Authorization 헤더를 확인하여 Bearer <token> 형식인지 검사한다. 헤더가 없거나 형식이 올바르지 않은 경우 401 TOKEN_REQUIRED 오류를 반환한다.

헤더 형식이 올바르면 토큰을 추출하여 jwt.verify()로 검증한다. 이때 HS256 알고리즘과 서버 비밀키를 사용하여 서명을 확인하고, 토큰의 만료 시간과 발급자 및 대상 서비스도 검사한다.

이후 sub, exp, iss, aud 등 필수 클레임의 자료형과 유효성을 확인한다. sub 에 저장된 사용자 ID 가 안전한 양의 정수인지 검사하고, findPublic()을 통해 실제 존재하는 사용자인지도 확인한다.

모든 검증에 성공하면 user.id 를 반환하여 이후 API 처리에 사용한다. 검증된 사용자 ID 를 다음 처리 과정에 전달하는 부분은 요청 주체 연결점에 해당한다.

오류 발생 시에는 다음과 같이 처리한다.

- 토큰 누락 또는 Bearer 헤더 형식 오류: 401 TOKEN_REQUIRED

- 서명 불일치, 잘못된 클레임 또는 존재하지 않는 사용자: 401 INVALID_TOKEN
- 토큰 유효 시간 만료: 401 TOKEN_EXPIRED

이를 통해 단순히 JWT 내용을 읽는 것이 아니라, 서버에서 토큰의 유효성을 검증한 사용자만 보호된 API 에 접근하도록 구현하였다.

3 4 작성자 권한 검사

파일 / 함수: **backend-express/src/auth.js / assertOwner()**

코드 발췌

```
export function assertOwner(userId, authorId) {
  if (userId !== authorId) {
    throw new ApiError(
      403,
      "FORBIDDEN",
      "본인이 작성한 게시글만 수정하거나 삭제할 수 있습니다."
    );
  }
}
```

동작과 응답 코드 설명

assertOwner() 함수는 인증된 사용자와 게시글 작성자가 동일한지 확인하는 역할을 한다.

userId 는 JWT 검증을 통해 확인한 로그인 사용자의 ID 이며, authorId 는 게시글에 저장된 작성자의 ID 이다. 두 값을 비교하여 일치하지 않으면 403 FORBIDDEN 오류를 발생시킨다.

이 과정은 인증된 사용자 ID 와 게시글 작성자 ID 를 비교하는 권한 경계 대조에 해당한다.

인증과 권한 검사는 서로 구분된다. JWT 가 없거나 유효하지 않은 경우에는 사용자 인증에 실패하므로 401 오류가 발생한다. 반면 JWT 가 유효하더라도 다른 사용자가 작성한 게시글을 수정하거나 삭제하려는 경우에는 403 오류가 발생한다.

프론트엔드에서 다른 사용자의 게시글에 수정·삭제 버튼을 표시하지 않더라도, 사용자가 직접 API 요청을 전송할 수 있으므로 서버에서 별도로 작성자 권한을 검사하도록 구현하였다.

실제 테스트에서 Bob 계정으로 Alice 가 작성한 게시글의 수정과 삭제를 시도하였을 때 403 FORBIDDEN 응답이 반환되는 것을 확인하였다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경

- 운영체제: Windows 11
- 런타임: Node.js v24.21.0
- 프론트엔드: React + Vite
- 백엔드: Node.js + Express
- 데이터베이스: SQLite
- 프론트엔드 접속 주소: http://localhost:4173
- 백엔드 API 주소: http://localhost:8000

테스트 기준 코드

GitHub 저장소: https://github.com/lsanghyl/JWT-Assignment

Commit SHA: 4427582fc6a130260b812750725586819e19df94

테스트 수행 내용

JWT 인증 기능에 대한 자동 테스트를 실행한 결과, BE-01 테스트 23 개가 모두 통과하였다. 또한 프론트엔드 FE-04 UI 테스트 5 개가 모두 통과하였다.

인증 기능 검증에서는 토큰이 없는 요청에 401 TOKEN_REQUIRED, 변조된 토큰에 401 INVALID_TOKEN, 만료된 토큰에 401 TOKEN_EXPIRED 가 반환되는 것을 확인하였다.

작성자 권한 검사에서는 Bob 계정으로 Alice 가 작성한 게시글의 수정 및 삭제를 요청하였으며, 두 요청 모두 403 FORBIDDEN 응답을 반환하였다.

프론트엔드는 npm run build 로 빌드한 후 npm run preview 를 실행하여 4173 번 포트에서 접속하였다. 게시글 목록 조회, 로그인, 게시글 작성 및 조회가 정상적으로 동작하였다.

또한 백엔드를 재실행한 이후에도 기존 게시글이 유지되는 것을 확인하여 SQLite 데이터 영속성을 검증하였다.

추가 확인 사항

공식 테스트 시나리오 T01~T17 의 판정과 증거는 각 테스트의 실제 실행 결과를 기준으로 별도 기록한다. 아직 실행 여부가 확인되지 않은 항목은 PASS 로 처리하지 않는다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인 목록·상세 조회	200	공개 목록 및 상세 조회 정상	PASS	그림 1

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T02	Alice 로그인	200	로그인 및 JWT 발급 정상	PASS	그림 2
T03	비밀번호 오류·없는 이메일	401 / INVALID_CREDENTIALS	비밀번호 오류 및 존재하지 않는 이메일 모두 401 / INVALID_CREDENTIALS 반환	PASS	그림 3
T04	무토큰 /me 및 글 작성	401 / TOKEN_REQUIRED	/api/me 조회 및 게시글 작성 요청 모두 401 / TOKEN_REQUIRED 반환	PASS	그림 4
T05	Alice 글 작성 및 /me	201, /me 200	게시글 작성 시 201 및 authorId=1 반환, /api/me 조회 시 200 및 Alice 정보 반환	PASS	그림 5
T06	Alice 본인 글 수정·삭제	200, 204	PUT 200 및 수정 내용 반영, DELETE 204 및 응답 본문 없음, 삭제 후 GET 404 확인	PASS	그림 6
T07	Bob 이 Alice 글 수정	403 / FORBIDDEN	PUT 403 / FORBIDDEN	PASS	그림 2
T08	Bob 이 Alice 글 삭제	403 / FORBIDDEN	DELETE 403 / FORBIDDEN	PASS	그림 2
T09	변조된 JWT 사용	401 / INVALID_TOKEN	GET POST PUT DELETE 모두 401 / INVALID_TOKEN 반환, 기존 데이터 변경 없음	PASS	그림 7
T10	만료된 JWT 사용	401 / TOKEN_EXPIRED	만료 JWT 요청 시 401 / TOKEN_EXPIRED 반환, 프론트엔드 만료 안내 및 로그인 상태 해제 확인	PASS	그림 8, 그림 9

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T11	로그아웃 후 /me	401 / TOKEN_REQUIRED	로그아웃 후 로그인 상태 해제, Authorization 헤더 없이 /api/me 요청 시 401 / TOKEN_REQUIRED 반환	PASS	그림 10, 그림 11
T12	authorId·userId 위조	400 / BAD_INPUT	authorId 및 userId 위조 요청 모두 400 / BAD_INPUT 반환, 위조 게시글 생성되지 않음	PASS	그림 12
T13	제목·본문 유효성 검사	400 / BAD_INPUT	빈 제목, 공백 내용, 길이 초과 요청 모두 400 / BAD_INPUT 반환, 게시글 생성되지 않음	PASS	그림 13
T14	없는 글 및 잘못된 ID	404 / NOT_FOUND, 400 / BAD_INPUT	존재하지 않는 ID 의 GET PUT DELETE 요청은 404 / NOT_FOUND, 잘못된 ID 형식의 GET 요청은 400 / BAD_INPUT 반환	PASS	그림 14
T15	서버 재실행 후 데이터 유지	기존 데이터 유지	서버 재시작 후 게시글 ID, 제목, 내용, 작성자 및 생성·수정 시각 모두 동일	PASS	그림 15
T16	새로고침 후 로그인 초기화	메모리 토큰 제거	브라우저 새로고침 후 로그인 상태 초기화, 기존 공개 게시글 3 개 유지	PASS	그림 16

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T17	무토큰 수정·삭제 및 Bob 본인 CRUD	401, 200·201·204	무토큰 PUT DELETE 요청 모두 401 / TOKEN_REQUIRED 반환. Bob 의 게시글 작성 201, 수정 200, 삭제 204, 삭제 후 조회 404 확인	PASS	그림 17, 그림 18

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	200	200 OK	T02, T05, T06
새 글 등록	201 및 Location	201 Created 확인, Location 헤더 추가 확인 필요	T05, T17
본인 글 삭제	204 및 빈 본문	204 No Content, 본문 길이 0	T06, T17
입력 오류·추가 작성자 필드	400 BAD_INPUT	400 / BAD_INPUT	T12, T13
잘못된 계정	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	T03
토큰 없음·헤더 형식 오류	401 TOKEN_REQUIRED	토큰 없음: 401 / TOKEN_REQUIRED, 헤더 형식 오류 별도 확인 필요	T04, T11, T17
변조·클레임 오류	401 INVALID_TOKEN	변조 토큰: 401 / INVALID_TOKEN, 클레임 오류 별도 확인 필요	T09
정상 서명의 만료 토큰	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED	T10
다른 작성자의 변경	403 FORBIDDEN	403 / FORBIDDEN	T07, T08
없는 글	404 NOT_FOUND	404 / NOT_FOUND	T14

보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거

JWT 없이 보호된 API 에 접근하거나 변조·만료된 JWT 를 전달했을 때 HTTP 401 Unauthorized 가 반환되는 것을 확인하였다. 무토큰 요청에서는 TOKEN_REQUIRED, 변조된 JWT 에서는 INVALID_TOKEN, 만료된 JWT 에서는 TOKEN_EXPIRED 오류 코드가 반환되었다.

다만 WWW-Authenticate: Bearer 응답 헤더의 실제 포함 여부는 별도로 확인하지 않았으므로 추가 검증이 필요하다.

401 과 403 에서 화면이 각각 어떻게 달라지는지 설명

HTTP 401 은 인증 정보가 없거나 유효하지 않은 경우에 발생하며, HTTP 403 은 정상적으로 인증된 사용자에게 요청한 작업의 권한이 없는 경우에 발생한다.

프론트엔드에서는 JWT 만료로 401 응답이 발생했을 때 '토큰이 만료되었습니다.'라는 안내 메시지가 표시되고 로그인 상태가 해제되는 것을 확인하였다. 반면 Bob 이 Alice 의 게시글을 수정·삭제하려고 시도했을 때는 서버에서 403 FORBIDDEN 이 반환되었다. 프론트엔드에서는 다른 작성자의 게시글에 수정·삭제 버튼을 제공하지 않으므로, 해당 권한 오류는 PowerShell 에서 직접 API 를 요청하여 검증하였다.

실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과

Bob 계정으로 Alice 의 게시글(ID: 4)에 수정 및 삭제 요청을 수행한 결과, 두 요청 모두 HTTP 403 Forbidden 과 FORBIDDEN 오류 코드가 반환되었다. 이후 공개 조회를 통해 해당 게시글이 유지되고 있음을 확인하였다.

또한 변조된 JWT 를 사용한 수정·삭제 요청에서도 HTTP 401 Unauthorized 와 INVALID_TOKEN 이 반환되었으며, 기존 게시글의 제목(alice write), 내용(aa), 작성자 ID(1)가 유지됨을 확인하였다. 이를 통해 인증 또는 권한 검증에 실패한 요청이 기존 데이터를 변경하지 못함을 검증하였다.

토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과

정상적인 JWT 를 사용하여 존재하지 않는 게시글(ID: 999999)에 수정 요청을 수행한 결과, HTTP 404 Not Found 와 NOT_FOUND 오류 코드가 반환되었다.

토큰 없이 존재하지 않는 게시글을 수정하는 요청에 대해서는 동일 조건의 직접 검증을 아직 수행하지 않았다. 따라서 인증 실패가 게시글 존재 여부 검사보다 우선하는지 확인하기 위해 추가 테스트가 필요하다.

테스트 증거

테스트 증거

테스트 수행 과정에서 수집한 API 응답 및 웹 브라우저 실행 화면은 원본 해상도와 가독성을 유지하기 위해 별도의 이미지 파일로 정리하였다. 각 증거는 테스트 ID 와 검증 항목에 따라 구분하였다.

로그인과 토큰 전송

증거 1. 로그인 성공 및 JWT 인증 요청 (T02, T05)

Alice 계정으로 로그인한 후 POST /api/auth/login 요청에서 HTTP 200 OK 가 반환되는 것을 확인하였다. 또한 JWT 를 이용한 GET /api/me 요청에서도 HTTP 200 OK 와 Alice 사용자 정보가 반환되었다. JWT 원문은 제출 화면에서 가린다.

증거 파일: 그림 1

다른 작성자의 수정과 삭제

증거 2. 타인 게시글 수정·삭제 권한 검증 (T07, T08)

Bob 계정으로 Alice 가 작성한 게시글(ID: 4)에 PUT 및 DELETE 요청을 수행한 결과, 두 요청 모두 HTTP 403 Forbidden 과 FORBIDDEN 오류 코드가 반환되었다. 이를 통해 게시글 작성자에 따른 접근 권한 제한이 정상적으로 동작함을 확인하였다.

증거 파일: 그림 2

토큰 변조와 만료

증거 3. JWT 변조 및 만료 검증 (T09, T10)

JWT 의 서명 일부를 변조하여 사용자 정보 조회 및 게시글 작성·수정·삭제 요청을 수행한 결과, 모든 요청에서 HTTP 401 Unauthorized 와 INVALID_TOKEN 오류 코드가 반환되었다.

또한 JWT 만료 후 보호된 API 에 접근했을 때 HTTP 401 Unauthorized 와 TOKEN_EXPIRED 가 반환되었으며, 프론트엔드에서는 만료 안내 메시지 표시와 로그인 상태 해제가 정상적으로 이루어졌다.

증거 파일: 그림 7, 그림 8, 그림 9

자동 검사와 로컬 배포 증거

자동 검사 결과

백엔드 자동 검사 BE-01 에서 총 23 개 테스트가 통과하였으며, 프론트엔드 자동 검사 FE-04 에서도 총 5 개 테스트가 통과하였다. 이를 통해 주요 API 기능과 프론트엔드 동작을 자동화된 테스트로 확인하였다.

로컬 배포 및 데이터 유지 확인

React 프론트엔드를 빌드한 후 Vite Preview 를 이용하여 localhost:4173 에서 실행하였다. 프론트엔드와 백엔드 API 간의 통신이 정상적으로 이루어지는 것을 확인하였다.

또한 백엔드 서버를 종료하고 재시작한 후 기존 게시글을 재조회하여 게시글 ID, 제목, 내용, 작성자 정보 및 생성·수정 시각이 모두 동일하게 유지되는 것을 확인하였다.

증거 파일: 자동 검사 실행 로그, 빌드 성공 로그, 로컬 실행 화면, 그림 15

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: 사용

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3 건을 작성하세요. 사용이 1 건이면 그 1 건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
ChatGPT	JWT 인증 및 게시판 구현 보조, 오류 해결, 테스트 명령문 작성, 결과 분석, 보고서 작성 보조	backend-express/src/auth.js, frontend-react/src/api.js, 게시판 관련 코드, 보고서 01~05	코드 제안 및 수정 내용은 직접 실행·검증 후 반영. 테스트 결과와 보고서 설명은 실제 확인 결과를 기준으로 정리

대표 사례 1. JWT 인증 및 권한 검증 구현

도구 / 날짜: ChatGPT / 2026 년 10 월 28 일

요청한 내용: JWT 발급 및 검증, Bearer 토큰을 이용한 사용자 인증, 게시글 작성자에 따른 수정·삭제 권한 제한 구현

AI 가 제안한 내용: jsonwebtoken 을 이용한 JWT 발급·검증 함수와 Authorization 헤더 검사, 작성자 ID 비교를 통한 권한 검증 방식

실제 반영 위치: backend-express/src/auth.js 및 관련 API 코드

내가 확인하고 수정한 내용: 실제 로그인과 게시글 CRUD 요청을 실행하여 정상 인증 시 200, 토큰이 없을 때 401, 다른 작성자의 게시글을 수정·삭제할 때 403 이 반환되는 것을 확인하였다. 코드의 불필요한 주석을 정리하고 실제 구현 내용과 일치하도록 수정하였다.

검증 방법과 결과: T02, T04, T07, T08 테스트에서 JWT 인증 및 작성자 권한 검증이 정상적으로 동작함을 확인하였다.

대표 사례 2. JWT 오류 상황 테스트

도구 / 날짜: ChatGPT / 2026 년 10 월

요청한 내용: JWT 변조·만료 상황과 잘못된 입력에 대한 API 테스트 방법 및 PowerShell 요청 명령어 작성

AI 가 제안한 내용: 변조된 JWT 와 만료된 JWT 를 사용한 API 요청, 잘못된 작성자 필드 및 입력값을 전달하는 테스트 방법과 예상 HTTP 응답 코드

실제 반영 위치: 보고서 04. 테스트 결과 및 테스트 증거

내가 확인하고 수정한 내용: AI 가 제안한 요청을 실제 서버에서 실행하고 반환된 HTTP 상태 코드와 오류 메시지를 직접 확인하였다. 만료 전에는 정상 처리되었던 요청을 만료 후 다시 실행하여 결과를 구분하였다.

검증 방법과 결과: T09 에서 INVALID_TOKEN, T10 에서 TOKEN_EXPIRED, T12~T13 에서 BAD_INPUT 응답을 확인하였다. 실패한 요청으로 기존 데이터가 변경되지 않았는지도 확인하였다.

대표 사례 3. 테스트 결과 분석 및 보고서 작성

도구 / 날짜: ChatGPT / 2026 년 10 월

요청한 내용: 구현 코드의 주요 동작 설명, 자동 검사 결과 분석, 테스트 증거 및 보고서 문장 정리

AI 가 제안한 내용: JWT 인증 처리 흐름, 프론트엔드 API 요청 구조, HTTP 응답별 테스트 결과에 대한 설명과 보고서 구성

실제 반영 위치: 보고서 02. 구현 구조, 03. 주요 코드 설명, 04. 테스트 결과, 05. AI 활용과 참고 자료

내가 확인하고 수정한 내용: 실제 소스 코드와 테스트 결과를 비교하여 설명을 작성하였다. 확인되지 않은 Location 헤더, 일부 오류 우선순위 등의 결과는 검증 완료로 기재하지 않았다.

검증 방법과 결과: 백엔드 자동 검사 23 개, 프론트엔드 자동 검사 5 개 통과 결과를 확인하였다. 또한 localhost:4173 에서 프론트엔드를 실행하고 백엔드 API 연동 및 SQLite 데이터 유지 여부를 확인하였다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
웹서비스설계 과제 안내 및 보고서 작성 양식 (수업 자료)	보고서 전체	과제 요구사항, 제출 형식, 보고서 구성 기준
docs/05_테스트시나리오.md (프로젝트 제공 문서)	보고서 04	T01~T17 테스트 항목과 기대 응답 기준
프로젝트 소스 코드 (https://github.com/lsanghyl/JWT-Assignment)	보고서 02~04	실제 구현 코드 및 테스트 결과 확인

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

06. 이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

클라이언트가 전달한 authorId 를 그대로 신뢰하면 다른 사용자의 ID 를 넣어 게시글을 작성하는 작성자 위조 문제가 발생할 수 있다.

내 코드는 JWT 인증을 통해 서버에서 사용자 ID 를 확인하고, 해당 ID 를 게시글의 작성자로 사용한다. 또한 게시글 작성 요청에 authorId 또는 userId 와 같은 작성자 지정 필드가 포함되면 BAD_INPUT 오류를 반환하도록 처리하였다.

실제로 Bob 계정으로 authorId: 1 또는 userId: 1 을 포함한 게시글 작성 요청을 보냈을 때 HTTP 400 BAD_INPUT 이 반환되었으며, 작성자가 위조된 게시글은 생성되지 않았다(T12).

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

프론트엔드에서 수정·삭제 버튼을 숨기더라도 사용자가 PowerShell 이나 API 클라이언트를 이용하여 서버에 직접 요청을 보낼 수 있기 때문이다. 따라서 화면에서 버튼을 숨기는 것과 별개로 서버에서 권한을 검사해야 한다.

내 코드는 auth.js 의 assertOwner(userId, authorId) 함수를 사용하여 인증된 사용자 ID 와 게시글 작성자 ID 를 비교한다. 두 값이 다르면 HTTP 403 FORBIDDEN 을 반환하여 변경을 차단한다.

실제로 Bob 의 JWT 로 Alice 의 게시글을 수정하거나 삭제하려고 했을 때 모두 403 FORBIDDEN 이 반환되었으며, 기존 게시글이 유지되는 것을 확인하였다(T07, T08).

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 JWT 에 포함된 정보를 해석하는 기능으로, 토큰의 서명이 올바른지 검증하지 않는다. 반면 verify 는 토큰의 서명과 유효성을 검증하므로 인증 과정에서는 verify 를 사용해야 한다.

내 코드는 currentUserId(req) 함수에서 Authorization 헤더의 Bearer 형식을 확인하고, jwt.verify()를 이용해 JWT 의 서명, HS256 알고리즘, 발급자(issuer), 대상(audience), 만료 시간을 검증한다. 이후 사용자 식별 정보인 sub 클레임을 검사하고 실제 사용자 정보도 조회한다.

검증에 실패한 토큰은 INVALID_TOKEN 또는 TOKEN_EXPIRED 등의 오류로 처리한다. 실제로 JWT 의 서명을 변조했을 때 401 INVALID_TOKEN, 만료된 JWT 를 사용했을 때 401 TOKEN_EXPIRED 가 반환되었다(T09, T10).

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 프론트엔드가 메모리에 보관하던 JWT 와 로그인 사용자 정보를 제거하는 동작이다. 따라서 이후 요청에는 인증 토큰이 첨부되지 않는다.

토큰 만료는 JWT 에 설정된 유효 시간이 지나 서버에서 해당 토큰을 더 이상 인정하지 않는 상태이다. 만료된 토큰으로 요청하면 서버는 TOKEN_EXPIRED 를 반환하며, 프론트엔드는 로그인 상태를 해제한다.

새로고침은 프론트엔드의 JavaScript 실행 환경을 다시 시작한다. 내 코드는 로그인 정보를 브라우저 저장소가 아닌 모듈 메모리에 보관하므로, 새로고침하면 로그인 상태가 초기화된다. 다만 서버에 저장된 공개 게시글은 그대로 조회할 수 있다.

이미 복사된 JWT 는 로그아웃하거나 새로고침해도 서버에서 자동으로 폐기되지 않는다. 따라서 만료 전이고 서명이 유효하다면 다시 사용될 수 있다. 반면 만료된 토큰은 재사용할 수 없다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

가장 어려웠던 부분은 JWT 인증과 게시글 작성자 권한 검증을 구분하여 올바른 HTTP 응답을 반환하도록 구현하는 과정이었다.

처음에는 로그인 상태와 화면의 버튼 표시만 확인하는 것으로 충분하다고 생각할 수 있었지만, 실제로는 API 에 직접 요청을 보내는 경우까지 고려해야 했다. 특히 토큰이 없거나 변조된 경우의 401 응답과, 정상적으로 인증되었지만 다른 사용자의 게시글을 변경하려는 경우의 403 응답을 구분해야 했다.

이를 해결하기 위해 인증 단계에서는 currentUserId(req)로 JWT 를 검증하고, 게시글 변경 단계에서는 assertOwner()로 작성자 권한을 검사하도록 구현 내용을 점검하였다.

이후 PowerShell 을 이용하여 무토큰 요청, 변조 토큰 요청, Bob 의 Alice 게시글 수정·삭제 요청을 각각 실행하였다. 그 결과 401 TOKEN_REQUIRED, 401 INVALID_TOKEN, 403 FORBIDDEN 이 반환되었으며, 실패한 요청 이후에도 게시글이 변경되지 않은 것을 확인하였다.

현재 남은 문제와 다음에 개선할 점

현재 주요 기능과 자동 검사는 정상적으로 동작하지만, 일부 HTTP 응답 헤더 및 오류 처리 우선순위는 추가 검증이 필요하다.

향후에는 JWT 로그아웃 시 서버 측 토큰 무효화 기능을 추가하고, 새로고침 이후에도 안전하게 로그인 상태를 관리할 수 있도록 인증 구조를 개선하고 싶다. 또한 테스트를 자동화하여 인증 및 권한 검증 과정에서 발생할 수 있는 오류를 더욱 체계적으로 확인하고 싶다.